Coursework Description - OOP 26/27

COMP0213 – Object Oriented Programming for Robotics and AI

1. Introduction and Coursework Objectives

In this coursework, you are required to design and implement the robotic grasping training tool defined in Section 2 (Grasping simulating in PyBullet), developing it as a delivery-ready software application. The software must be implemented following the Model-View-Controller (MVC) architectural pattern and use PyQt6, as described in Section 3 (MVC Architecture Requirements).

You will need to apply the Object-Oriented Programming (OOP) concepts and Python libraries studied during weeks 1-6 of the module, encapsulated by a software architecture and interface, using software engineering methods approached during week 7-9.

In your report, you should explain and justify the design decisions made when separating the system into the corresponding MVC layers, highlighting how responsibilities are distributed and how the architecture improves maintainability, scalability, and software quality.

Intended Leaning Outcomes (ILO):

- Having hands-on experience in grasping simulation with PyBullet
- Practical experience with Machine Learning libraries such as ScikitLearn by classifying good/bad grasps
- Data visualization by plotting results using libraries such as matplot/seaborn
- OOP concepts being applied into robotics problems
- MVC architecture software development
- Mastering Graphical User Interface with permission control

Groups

Groups of maximum 4 students will be formed during the sessions. Marks will be the same for all participants, unless uncommon situations are brough to the Module Leads.

2. Grasping simulating in PyBullet

Task Context: Grasping & Grasp Planning in Robotics and AI

In 2024, the new World Robotics report recorded 4,281,585 units operating in factories worldwide. Although it is hard to give exact numbers to the type of robots comprised in this sum, industrial robots are generally robotic arms capable of operating autonomously for soldering, screwing, and moving through grasping[1].

Grasping refers to the act of a robotic end-effector (e.g., gripper or hand) securely holding to manipulate an object. It's a fundamental capability for physical interaction with the environment—whether it's picking up a cup, assembling components, or assisting users with disabilities.

Grasp planning involves computational strategies that determine how, where, and when to grasp. It integrates sensory input, object geometry, contact models, and robot kinematics and control to construct and evaluate grasp candidates and select the most stable, efficient one using analytic, data-driven or hybrid algorithms.

Importance in Robotics and AI:

- **Physical Autonomy**: Enables robots to perform tasks in unstructured environments, such as homes, hospitals, or disaster zones.
- **Human-Robot Interaction (HRI)**: Safe and adaptive grasping fosters trust and usability, especially in educational or assistive robotics.
- **Learning & Adaptation**: Data-driven methods such as Reinforcement learning and imitation learning allow AI systems to generalize grasps and manipulation strategies across novel objects.
- **Ethical Impact**: Designing inclusive grasp strategies ensures accessibility for diverse user needs—children, elderly individuals, or users with mobility impairments.

Therefore, grasp synthesis, namely grasp configuration generation is an important step for an autonomous robot to act in human environments. In this process, two of the main steps involve selecting candidate grasp configurations, i.e. how to position the robotic hand with respect to an object and how to quantify the quality of a grasp configuration (meaning whether the grasping with that configuration would be successful, leading to successful further manipulation). These two steps enable robots to efficiently and safely anticipate how to interact with diverse objects and users, enhancing autonomy, adaptability, and social acceptance in human-robot environments.

[1] https://shorturl.at/d2cc0

Objective: Build a full pipeline of generating grasp candidates/configurations and binary scoring each grasp candidate using ML models and PyBullet simulation.

Breakdown of the Grasping component of the coursework

In this project, you will be provided with models of robotic grippers and objects to implement a grasp planning framework. An example of a three-fingered gripper is shown in Figure1. The objective is to design and evaluate a grasp planning pipeline consisting of two main components:

1. A grasp configuration sampling strategy.
2. A classifier for evaluating grasp success.

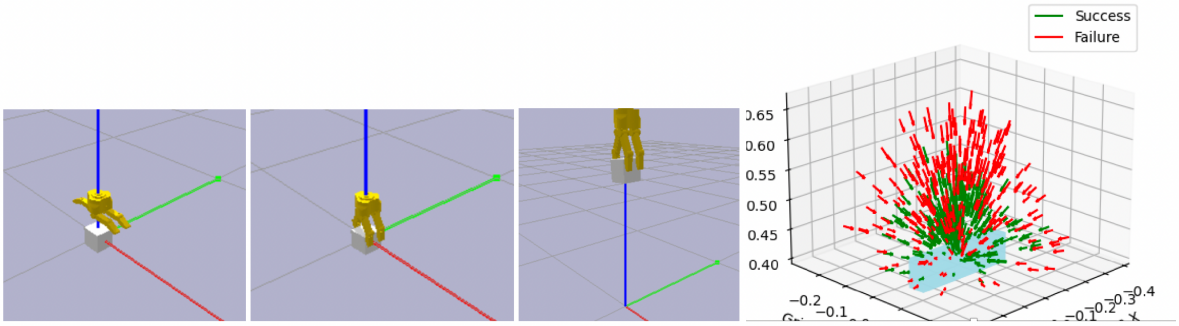


Figure 1- Left: The gripper approaching the cube to grasp (leftmost), grasping the cube (middle) at the planned pose, and lifting and holding the cube lifted (rightmost). This is an example of a successful grasp. Right: Example sampled hand positions and orientations/approach vectors around an object.

The grasp planning procedure will operate as follows: candidate grasp poses, defined as possible placements of the gripper relative to an object, will be generated using a sampling strategy. Each candidate grasp will then be executed by positioning the gripper at the sampled pose, closing the fingers around the object, and lifting the object to a specified height.

To execute the grasps, in cases where collisions with the object are frequent, it is recommended to first position the gripper at an approach pose using the planned gripper orientation. This approach pose is defined at a specified distance from the intended final grasp pose, ensuring that the gripper fingers can open without colliding with the object. The execution procedure is as follows: first, open the fingers of the gripper; then move the hand to the approach pose; finally, advance the gripper along the approach trajectory to reach the planned grasp pose and perform the grasp.

During lifting test, If the object remains securely in the gripper for several seconds without slippage or dropping, the grasp will be labelled as stable. This procedure will be repeated for each sampled grasp in order to obtain binary success labels. The feature space of the classifier can be six-dimensional, consisting of three translational and three rotational parameters describing the hand pose relative to the object. Alternatively, a seven-dimensional feature space may be employed if orientation is represented using quaternions. In addition, you can use other informative data features such as contact points per finger and test which feature combination leads to best results.

To construct the dataset, you are required to employ at least two distinct grippers and two distinct objects. The implementation should adhere to the principles of OOP to ensure extensibility with respect to different gripper and object models. You need to design your classes to ensure you do not repeat code to achieve the goal of using two grippers and two objects, e.g. making use of inheritance.

The specific strategy for grasp pose sampling may be chosen at your discretion. One possible approach is to position the object at the center of a notional sphere and to generate grasp candidates by directing the gripper toward the object along radii of this sphere (see Figure 1). Variations in distance and the addition of Gaussian noise to the sampled poses can be incorporated to enhance dataset realism and increase data diversity.

During execution, the gripper pose relative to the object (measured prior to lifting) should be recorded. These poses will serve as input features to the classifier, while the binary grasp outcome (success or failure) will provide the target labels.

The final dataset must be organized as a pandas DataFrame, where the input dimensions (pose features) occupy the initial columns and the final column contains the corresponding ground-truth labels. This dataset will form the basis for training and evaluating the grasp classification model. You will work with a balanced dataset, where the number of positive and negative samples is equal. The dataset should contain at least 120–140 samples (for a dataset with 6-dimensional input, 20 samples per class per feature).

Building your classifier-based planner

Training Phase

Train a classification model of your choice to distinguish between successful and unsuccessful grasps using the dataset constructed in the previous stage. Employ a classifier available within the scikitlearn library and use the recorded grasp configurations (position and orientation) as input features. The classifier should be

trained to predict binary outcomes corresponding to grasp success or failure. You are encouraged to experiment with different classifiers and hyperparameter settings in order to identify the configuration that yields the highest validation accuracy, at least 60-70%.

Testing Phase

Construct a separate test set of grasp poses following the same sampling strategy used during training. Using the classifier trained in the previous phase, predict the success label of each test grasp and then evaluate the correctness of the prediction by executing the grasp: placing the gripper at the grasp pose, closing the fingers, and lifting the object. Test the classifier on at least ten grasp attempts and report the overall success rate of the predictions. Make sure the test data is not used in training.

Terms definitions

1) **Grasping**: The process by which the gripper closes its fingers around the target object to establish a secure hold.

2) **Grasp planning**: The determination of an appropriate gripper pose in space (defined by position x, y, and z, and orientation yaw, pitch, roll angles) where the gripper is placed to close its fingers around the object to safely secure it for further manipulation.
3) **Lifting**: The action of raising the grasped object to a specified height while maintaining the same grasp configuration.
4) **Successful grasp**: A grasp is considered successful if the object remains securely held by the gripper for more than three seconds after lifting.

Requirements

You will receive the code and report submission guidelines. Your task is to implement the project using Object-Oriented Programming (OOP) principles, such as encapsulation, inheritance, polymorphism, abstract classes, and UML class diagrams. Additionally, you should incorporate other libraries introduced in this course, such as NumPy, Pandas, and others, where applicable.

3. MVC Architecture Requirements

MVC (Model-View-Controller) is a software design pattern that divides an application into three interconnected parts to separate data handling (Model), user interface (View), and user input (Controller), as briefly explained to follow.

Model

The Model layer should handle all data management, business logic, and core application functionality. It should be responsible for:

- Managing users and authentication data.
- Storing and retrieving grasp records.
- Managing machine learning models and training results.
- Generating reports on e.g. dataset statistics.
- Handling database operations and data persistence.
- Defining business rules and contracts.

View

The View layer should provide all graphical user interfaces and user interaction components. It should:

- Display data to users, e.g. plot the good and bad grasp configurations, validation performance
- Collect user input, e.g. number of grasps to be generated, logging specifications.
- Present notifications, warnings, and status messages.
- Remain independent from business logic.

Controller

The Controller layer should act as an intermediary between the Model and the View. It should:

- Process user actions and events, such as running the collection of data with a given type of grasp, object or gripper.
- Validate user requests.
- Invoke the appropriate Model methods (evoking the right methods).
- Update the View with the results of operations (showing the user the processed result in the correct way).
- Coordinate communication between layers while maintaining separation of concerns.

Functional Requirements

When the application starts, the user should be presented with a main menu containing the following options:

Main Menu

- **Login**
- **About the Program and Developers**
- **Frequently Asked Questions (FAQ)**

Logged-In User Features

After successful authentication, users should have access to the following functionalities:

- Use the simulator to generate new grasp entries.
- View previously stored grasp records.
- View machine learning models that have already been trained.
- Train new machine learning models.
- Generate and view reports, such as the number of entries in the dataset, dataset statistics, users' activities, system logs.
- Manage and review their own generated data and results.

All actions performed within the system must be associated with the authenticated user who created them. This includes:

- Generated grasps.
- Trained machine learning models.
- Generated reports.
- Any additional user-created content.

Software Engineering Requirements

Exception Handling

All exceptions and errors should be handled elegantly and professionally. The implementation should ensure that:

- System failures are communicated clearly to the user.
- Layers remain independent and loosely coupled.

- Error handling does not violate the MVC architecture.
- Components can be integrated with similar layers from other software systems if required.

User Access Management

User authentication and access control must be implemented following the approaches covered during the practical sessions. The system should support:

- Secure login procedures.
- User-specific data ownership.
- Appropriate access restrictions where necessary.

Interfaces

You should propose the best GUI windows to organise your software and discuss your decisions on your report. You also should consider inclusion and accessibility[2] criteria when developing your interface and justifies it for the chosen components (Dropbox, Buttons, Text areas, etc). There are no limit numbers for windows but consider software optimisation in terms of redundancy and user experience, such as usability, cognitive load, learning curve, and so on.

Bonus Marks - **Deadline to propose your idea: week 8 by the lab session**

Earn extra marks by implementing additional features. Proposals must be approved by the **Week 8 lab session**. Example ideas are listed below.

For Grasp planning part:

Additional marks may be awarded implementing extra features such as using an integrated arm with a motion planner, using a state-of-the-art Machine Learning model-based grasp planner and analysing its results for comparison, using point cloud inputs, rigorous analysis of the results e.g. ROC curves, studying the success rate with respect to data size, Using more input features than hand poses, e.g.contact points, hand joint angles (in case of the 3-fingered hand), dimensionality reduction for high dimensional input features.

For MVC part:
Additional marks may be awarded for implementing extra features such as a **chatbot assistant** within the application. The chatbot could be used to:

[2] https://www.accessibilitychecker.org/blog/inclusive-design/

- Support users in navigating the system.
- Answer common questions.
- Provide guidance on system features.
- Offer help with grasp generation, model training, and report interpretation.
- Decisions made and the reasoning
    - E.g. “We chose implementing method “foo” in the “View” module, as showed in the UML diagram page 2, of the MCV due to security optimisation ….”

4. References:

1. PyBullet website: https://pybullet.org/

https://github.com/bulletphysics/bullet3

https://github.com/bulletphysics/bullet3/tree/master/docs

2. PyBullet Quick start pdf file included in Moodle.

https://github.com/bulletphysics/bullet3/blob/master/docs/pybullet_quickstartguide.pdf